```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf
from sklearn.model_selection import KFold

sns.set_theme(style="whitegrid")
```

Q1

Double-click (or enter) to edit

1. A model is linear if it is linear in the coefficients/parameters. this does not mean neccessarily linear in the variables themselves.
2. For a dummy variable variable, the coefficient measures the difference between that category and the omitted reference group, holding other variables fixed. if you keep the intercept only one category must be dropped. Further more the intercept is the mean for hte reference group and the dummy coefficient is how much higher or lower the other group is relative to that reference group.
3. Linear regression can technically be used for classification, but it is usually not ideal because predicted values can fall below 0 or above 1, and the relationship between predictors and class probability is usually not truly linear. Logistic regression is better at classification than linear.
4. Overfitting can be found by very good performance on the training data but noticeably worse performance on validation or test data, unstable coefficients, and a model that becomes too sensitive to noise or small changes in the sample.
5. multicollinearity means one explanatory varaible is highly correlated with another, so it becomes hard to isolate each variable's seperate effect. Like in two stage least squares setting, if your endogenous regressor is highly collinear with other controls, the first stage has trouble seperating unique variation,

which makes the second stage coefficient estimates unstable and not so accurate.

6. It is possible to include non linear relationships by adding transformation of x, such as x^2, x^3, logs or splines. The model can still be linear in coefficients even if the variables are transformed.
7. the slope coefficient gives the expected change in y from a one unit increase in x, holding other variables constant.
8. A train/test split uses one random split of the data into training and testing sets. It is simple and fast, but results can depend on the specific split. k fold cross validiation repeatdly splits the data into folds and avergages perfromance acrooss all of them. So it gives a more stable estimate of out of sample performance.
9. the value of k is usually chosen as a balance between computation and stability, such as common choices are like 5 or 10. Very common because it usually gives a reliable estiminate without being too compuationally expensive.

Q2.1

```
q2 = pd.read_csv("Q1_clean.csv")
q2 = q2.rename(columns={
    "Neighbourhood ": "Neighbourhood",
    "Review Scores Rating": "Review_Scores_Rating"
})

q2.head()
```

	Price	Review_Scores_Rating	Neighbourhood	Property Type	Room Type
0	549	96.0	Manhattan	Apartment	Private room
1	149	100.0	Brooklyn	Apartment	Entire home/apt
2	250	100.0	Manhattan	Apartment	Entire home/apt
3	90	94.0	Brooklyn	Apartment	Private room
4	270	90.0	Manhattan	Apartment	Entire home/apt

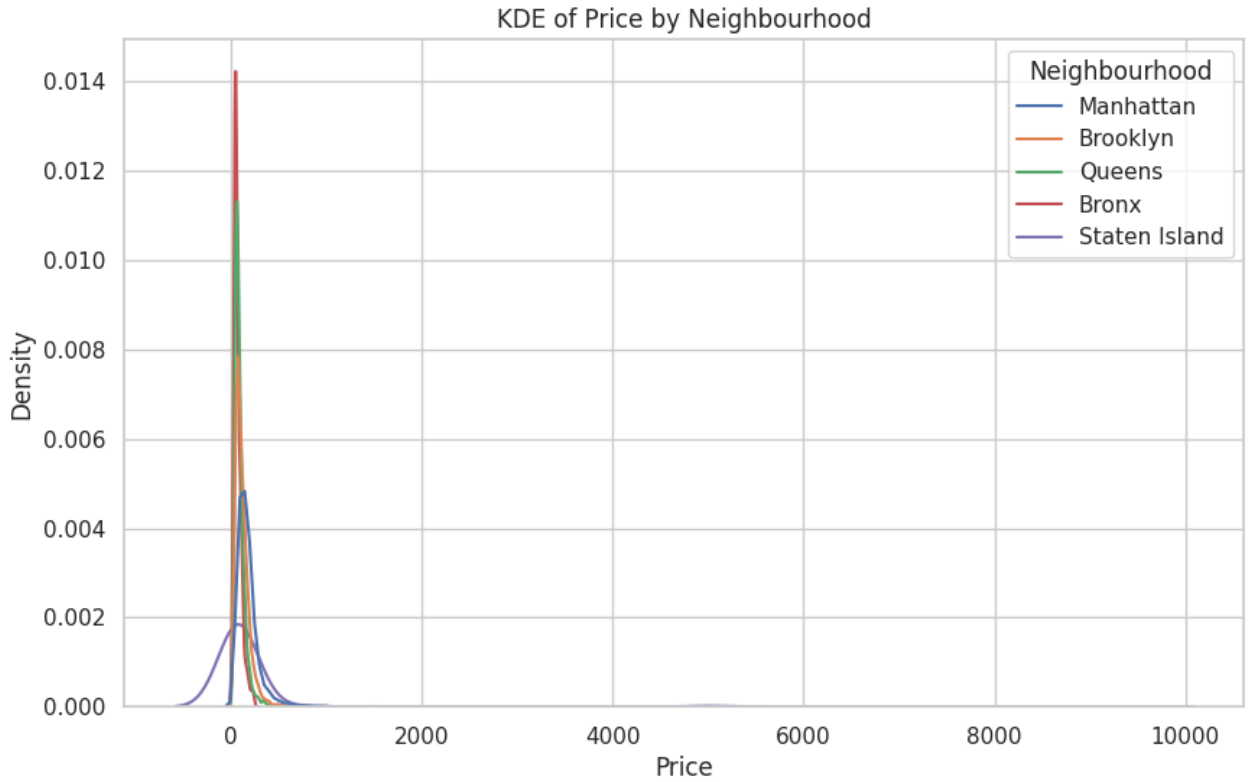
```
avg_table = q2.groupby("Neighbourhood")[["Price", "Review_Scores_Rati
avg_table
```

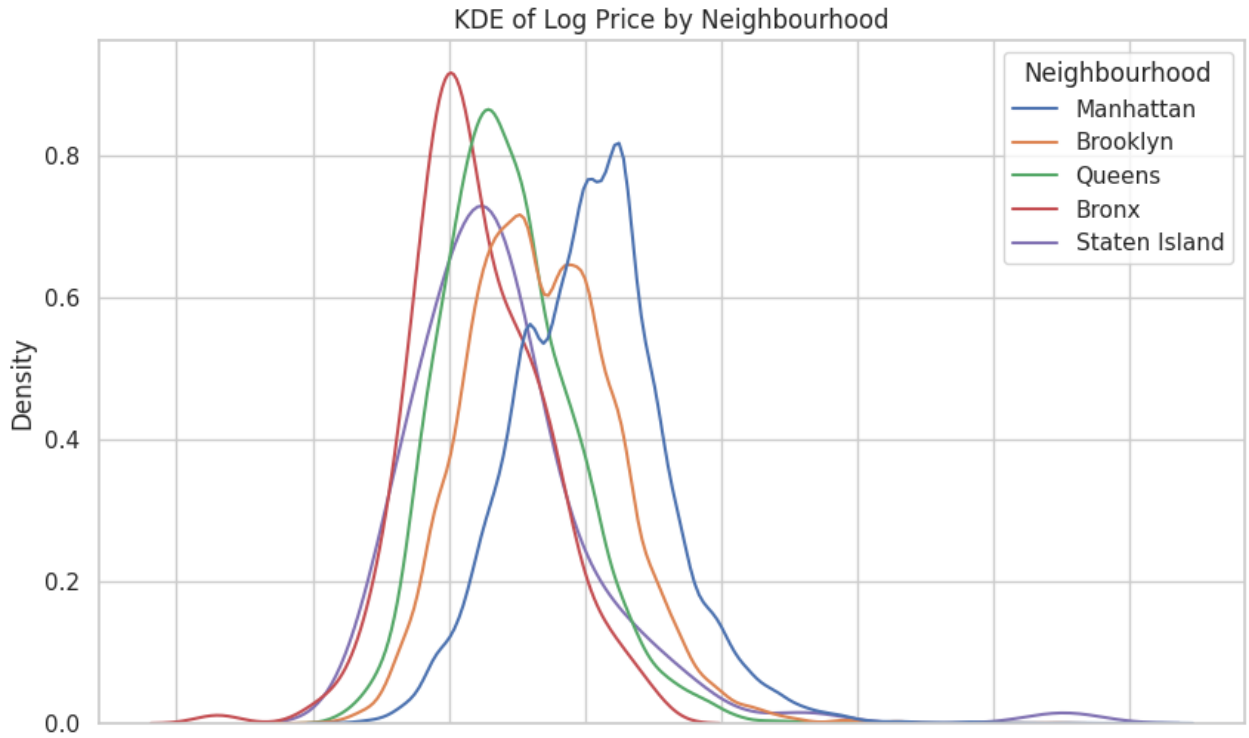
Neighbourhood	Price	Review_Scores_Rating
Manhattan	183.664286	91.801496
Staten Island	146.166667	90.843750
Brooklyn	127.747378	92.363497
Queens	96.857233	91.549057
Bronx	75.276498	91.654378

```
plt.figure(figsize=(10,6))
sns.kdeplot(data=q2, x="Price", hue="Neighbourhood", common_norm=Fals
plt.title("KDE of Price by Neighbourhood")
plt.show()
```

```
q2["Log_Price"] = np.log(q2["Price"])

plt.figure(figsize=(10,6))
sns.kdeplot(data=q2, x="Log_Price", hue="Neighbourhood", common_norm=F
plt.title("KDE of Log Price by Neighbourhood")
plt.show()
```





Manhattan is the most expensive on average

Double-click (or enter) to edit

```
avg_table
```

Neighbourhood	Price	Review_Scores_Rating
Manhattan	183.664286	91.801496
Staten Island	146.166667	90.843750
Brooklyn	127.747378	92.363497
Queens	96.857233	91.549057
Bronx	75.276498	91.654378

Q2.2

```
model_q22 = smf.ols("Price ~ C(Neighbourhood)", data=q2).fit()
print(model_q22.summary())
print(model_q22.params)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Price   R-squared:
Model:                            OLS   Adj. R-squared:
Method:                 Least Squares   F-statistic:
Date:                Thu, 26 Mar 2026   Prob (F-statistic):          4
Time:                        04:42:34   Log-Likelihood:            -1.
No. Observations:               22153   AIC:                         2
Df Residuals:                   22148   BIC:                         2
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                                        coef    std err          t
------------------------------------------------------------------------------
Intercept                            75.2765      9.868      7.628
C(Neighbourhood)[T.Brooklyn]         52.4709      9.994      5.250
C(Neighbourhood)[T.Manhattan]       108.3878      9.959     10.884
C(Neighbourhood)[T.Queens]           21.5807     10.520      2.051
C(Neighbourhood)[T.Staten Island]    70.8902     17.819      3.978
==============================================================================
Omnibus:                    51232.379   Durbin-Watson:
Prob(Omnibus):                  0.000   Jarque-Bera (JB):       119214
Skew:                          22.206   Prob(JB):
Kurtosis:                    1138.591   Cond. No.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
Intercept                            75.276498
C(Neighbourhood)[T.Brooklyn]         52.470881
C(Neighbourhood)[T.Manhattan]       108.387789
C(Neighbourhood)[T.Queens]           21.580735
C(Neighbourhood)[T.Staten Island]    70.890169
dtype: float64
```

Because the model includes an intercept, one neighborhood is ommitted as the reference group. Here the intercept is the mean price for the ommitted group and each neighborhood coefficient is the difference between that neighborhood's mean adn the refrence groups mean.

Q2.3

```
model_q23 = smf.ols("Price ~ Review_Scores_Rating", data=q2).fit()
print(model_q23.summary())
print(model_q23.params)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Price   R-squared:
Model:                            OLS   Adj. R-squared:
Method:                 Least Squares   F-statistic:
Date:                Thu, 26 Mar 2026   Prob (F-statistic):           
Time:                        05:12:45   Log-Likelihood:            -1.
No. Observations:               22153   AIC:                         2
Df Residuals:                   22151   BIC:                         2
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                          coef    std err          t      P>|t|
------------------------------------------------------------------------------
Intercept              60.8784     10.423      5.841      0.000
Review_Scores_Rating    1.0208      0.113      9.051      0.000
==============================================================================
Omnibus:                    50149.155   Durbin-Watson:
Prob(Omnibus):                  0.000   Jarque-Bera (JB):       102558
Skew:                          21.086   Prob(JB):
Kurtosis:                    1056.240   Cond. No.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
Intercept              60.878391
Review_Scores_Rating    1.020827
dtype: float64
```

The slope on the review scores rating is about 1.02. this means that a 1 point increase in review is associated with about 1 dollar and 2 cents increase in nightly price, on average.

```
model_q24 = smf.ols("Price ~ C(Neighbourhood) + Review_Scores_Rating",
print(model_q24.summary())
print(model_q24.params)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Price   R-squared:
Model:                            OLS   Adj. R-squared:
Method:                 Least Squares   F-statistic:
Date:                Thu, 26 Mar 2026   Prob (F-statistic):          1
Time:                        05:14:07   Log-Likelihood:            -1.
No. Observations:               22153   AIC:                         2
Df Residuals:                   22147   BIC:                         2
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                                        coef    std err          t
------------------------------------------------------------------------------
Intercept                           -23.8126     14.105     -1.688
C(Neighbourhood)[T.Brooklyn]         51.7042      9.973      5.185
C(Neighbourhood)[T.Manhattan]       108.2287      9.938     10.891
C(Neighbourhood)[T.Queens]           21.6946     10.498      2.067
C(Neighbourhood)[T.Staten Island]    71.7666     17.781      4.036
Review_Scores_Rating                  1.0811      0.110      9.812
==============================================================================
Omnibus:                    51385.422   Durbin-Watson:
Prob(Omnibus):                  0.000   Jarque-Bera (JB):       121553
Skew:                          22.369   Prob(JB):
Kurtosis:                    1149.681   Cond. No.                   2
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
[2] The condition number is large, 2.26e+03. This might indicate that
strong multicollinearity or other numerical problems.
Intercept                           -23.812561
C(Neighbourhood)[T.Brooklyn]         51.704240
C(Neighbourhood)[T.Manhattan]       108.228737
C(Neighbourhood)[T.Queens]           21.694600
C(Neighbourhood)[T.Staten Island]    71.766552
Review_Scores_Rating                  1.081116
dtype: float64
```

Controlling for the neighborhood, the slope on review_scores_rating increases slightly from 1.02 to about 1.08. That means ratings still ahve a positve association with price, and that relationships become a little stronger once neighborhood differences are accounted for. The neighborhood coefficient now represents price differences across neighborhoods holding review score fixed.

```
model_q25 = smf.ols(
    "Price ~ C(Neighbourhood) + C(Neighbourhood):Review_Scores_Rating'
    data=q2
).fit()

print(model_q25.summary())
print(model_q25.params)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Price   R-squared:
Model:                            OLS   Adj. R-squared:
Method:                 Least Squares   F-statistic:
Date:                Thu, 26 Mar 2026   Prob (F-statistic):
Time:                        05:15:42   Log-Likelihood:            -1.
No. Observations:               22153   AIC:                         2
Df Residuals:                   22143   BIC:                         2
Df Model:                           9
Covariance Type:            nonrobust
==============================================================================
                                                          coef    std
------------------------------------------------------------------------------
Intercept                                              22.6384    110
C(Neighbourhood)[T.Brooklyn]                          -16.3930    111
C(Neighbourhood)[T.Manhattan]                          41.3154    111
C(Neighbourhood)[T.Queens]                             28.2597    115
C(Neighbourhood)[T.Staten Island]                    2963.0710    183
C(Neighbourhood)[Bronx]:Review_Scores_Rating            0.5743      1
C(Neighbourhood)[Brooklyn]:Review_Scores_Rating         1.3155      0
C(Neighbourhood)[Manhattan]:Review_Scores_Rating        1.3040      0
C(Neighbourhood)[Queens]:Review_Scores_Rating           0.5020      0
C(Neighbourhood)[Staten Island]:Review_Scores_Rating   -31.2574      1
==============================================================================
Omnibus:                    50765.713   Durbin-Watson:
Prob(Omnibus):                  0.000   Jarque-Bera (JB):       120119
Skew:                          21.680   Prob(JB):
Kurtosis:                    1142.942   Cond. No.                    1
==============================================================================
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
[2] The condition number is large, 1.8e+04. This might indicate that tl
strong multicollinearity or other numerical problems.
Intercept                                                 22.638385
C(Neighbourhood)[T.Brooklyn]                             -16.392990
C(Neighbourhood)[T.Manhattan]                             41.315429
C(Neighbourhood)[T.Queens]                                28.259701
C(Neighbourhood)[T.Staten Island]                       2963.071040
C(Neighbourhood)[Bronx]:Review_Scores_Rating               0.574311
C(Neighbourhood)[Brooklyn]:Review_Scores_Rating            1.315476
C(Neighbourhood)[Manhattan]:Review_Scores_Rating           1.304014
C(Neighbourhood)[Queens]:Review_Scores_Rating              0.502017
C(Neighbourhood)[Staten Island]:Review_Scores_Rating     -31.257437
dtype: float64
```

teh slopes are not exactly the same across neighborhoods. Brooklyn and amanhattan have fairly similar positive slopes, the bronx and queens have smaller positive slopes and staten island has a strange negative slope that is likely unstable because the sample size is smaller. So there is some evidence that the relationship between review score and price differs by neighborhood.

```
def cv_rmse(formula, data, k=10, seed=1):
    kf = KFold(n_splits=k, shuffle=True, random_state=seed)
    rmses = []

    for train_idx, test_idx in kf.split(data):
        train = data.iloc[train_idx]
        test = data.iloc[test_idx]

        model = smf.ols(formula, data=train).fit()
        pred = model.predict(test)
        rmse = np.sqrt(np.mean((test["Price"] - pred) ** 2))
        rmses.append(rmse)

    return np.mean(rmses)

formulas = {
    "Part 3: Price ~ Review_Scores_Rating": "Price ~ Review_Scores_Ra
    "Part 4: Price ~ C(Neighbourhood) + Review_Scores_Rating": "Price
    "Part 5: Price ~ C(Neighbourhood) + C(Neighbourhood):Review_Score
}

for name, formula in formulas.items():
    print(name, "RMSE =", cv_rmse(formula, q2))
```

```
Part 3: Price ~ Review_Scores_Rating RMSE = 142.46290619252565
Part 4: Price ~ C(Neighbourhood) + Review_Scores_Rating RMSE = 138.672
Part 5: Price ~ C(Neighbourhood) + C(Neighbourhood):Review_Scores_Rati
```

The model from part 4 preforms best in cross validation while the neighborhood only plus rating model ahs the lowest RMSE, while the interaaction model in part 5 does not improve enough to beat it. So part 4 seems like the best balance of fit and simple protrayal.

part 3: 142.46 part 4: 138.67 part 5: 139.66

Q3

```
cars = pd.read_csv("cars_hw.csv")
cars.head()
```

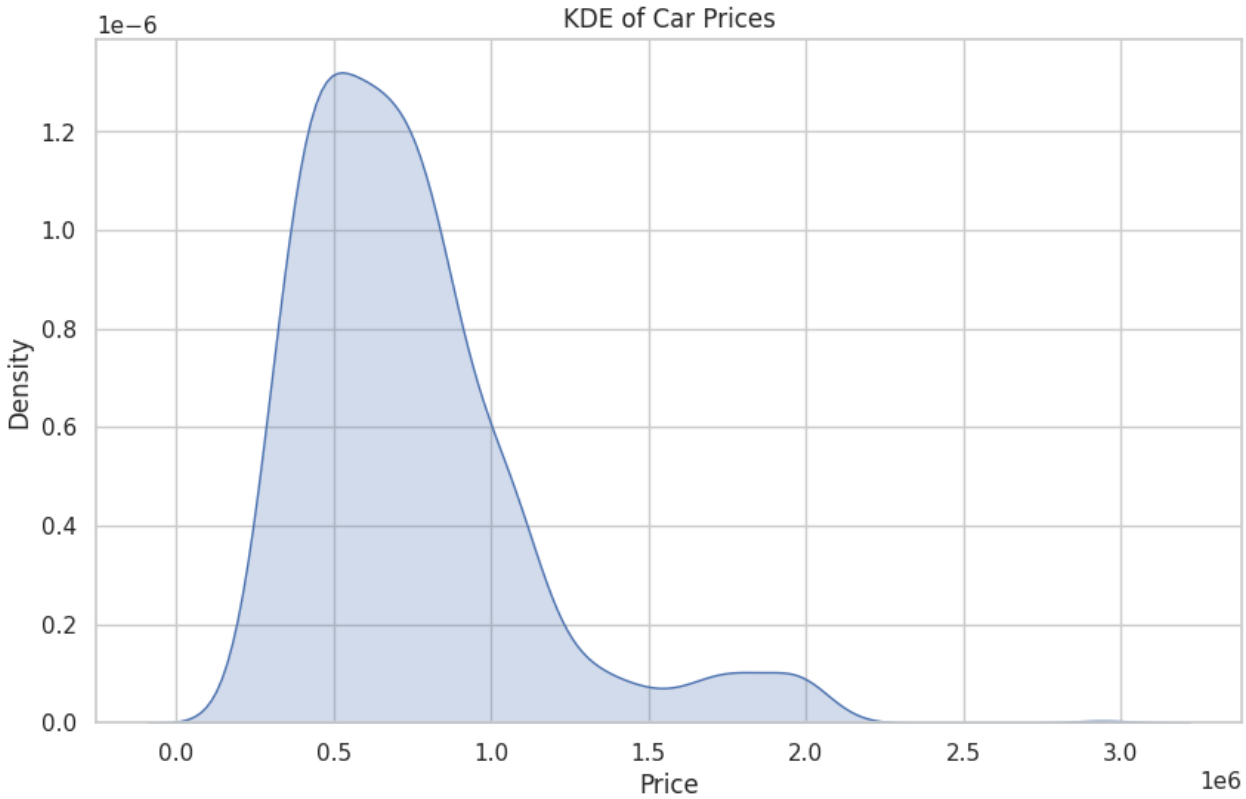
	Unnamed: 0	Make	Make_Year	Color	Body_Type	Mileage_Run	No_of_
0	1	Volkswagen	2017	silver	sedan	44611	
1	2	Hyundai	2016	red	crossover	20305	
2	3	Honda	2019	white	suv	29540	
3	4	Renault	2017	bronze	hatchback	35680	
4	5	Hyundai	2017	orange	hatchback	25126	

```
cars["Price"].describe()
```

	Price
count	9.760000e+02
mean	7.410195e+05
std	3.673234e+05
min	1.880000e+05
25%	4.757500e+05
50%	6.665000e+05
75%	8.830000e+05
max	2.941000e+06

dtype: float64

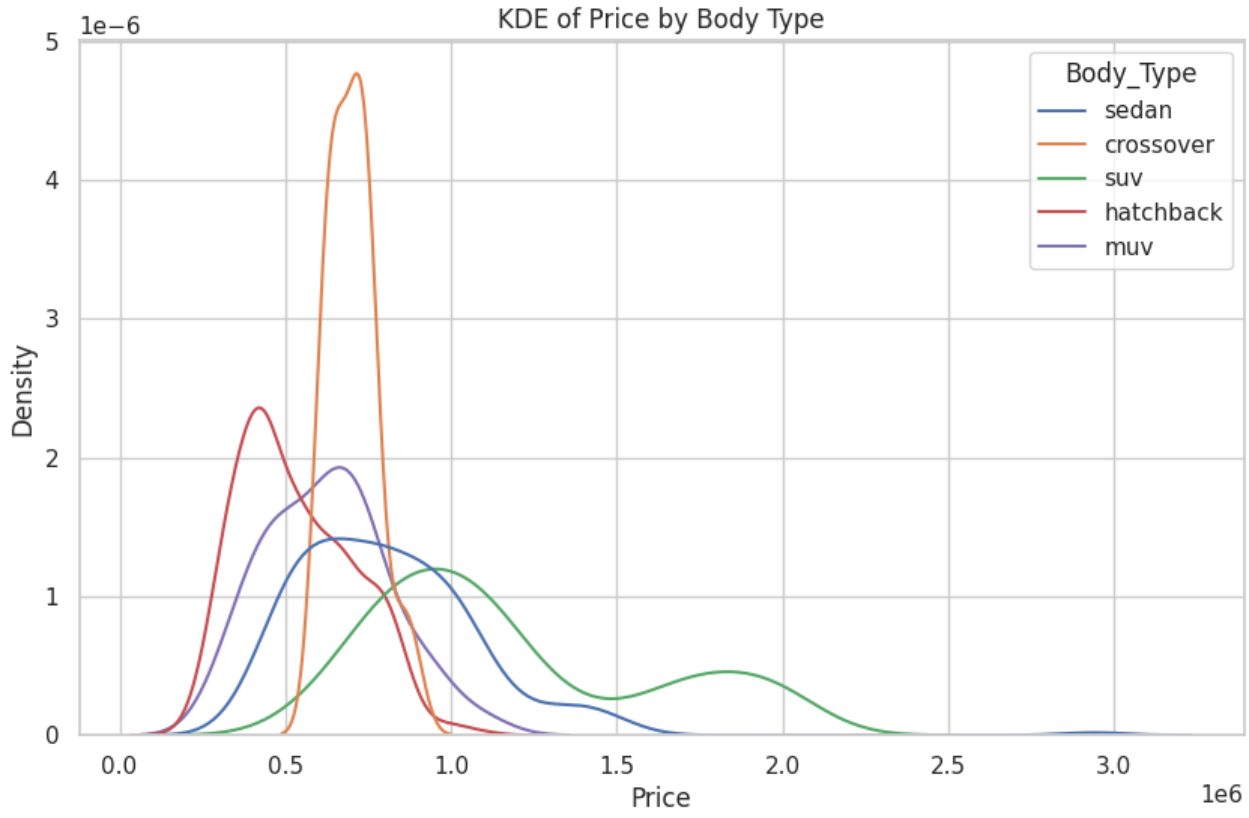
```
plt.figure(figsize=(10,6))
sns.kdeplot(data=cars, x="Price", fill=True)
plt.title("KDE of Car Prices")
plt.show()
```




```
body_summary = cars.groupby("Body_Type")["Price"].describe()
body_summary
```

Body_Type	count	mean	std	min	25%	50%
crossover	21.0	7.050952e+05	72849.780207	608000.0	638000.0	706000.0
hatchback	484.0	5.339773e+05	173086.729837	188000.0	401750.0	509000.0
muv	38.0	6.264211e+05	184093.488526	328000.0	489750.0	621500.0
sedan	227.0	8.097841e+05	286736.770092	368000.0	598000.0	772000.0
suv	206.0	1.176495e+06	417479.028217	559000.0	860000.0	1065000.0

```
plt.figure(figsize=(10,6))
sns.kdeplot(data=cars, x="Price", hue="Body_Type", common_norm=False)
plt.title("KDE of Price by Body Type")
plt.show()
```



SUVs are the most expensive on average which is then followed by sedans. SUVs also appear to have the most price variance with sedans showing substantial spread.

```
cars.groupby("Body_Type")["Price"].mean().sort_values(ascending=False
```

Body_Type	Price
suv	1.176495e+06
sedan	8.097841e+05
crossover	7.050952e+05
muv	6.264211e+05
hatchback	5.339773e+05

dtype: float64

```
cars.groupby("Body_Type")["Price"].std().sort_values(ascending=False)
```

Body_Type	Price
suv	417479.028217
sedan	286736.770092
muv	184093.488526
hatchback	173086.729837
crossover	72849.780207

dtype: float64

largest variance is SUV and sedan

Q3.2

```
model_q32_linear = smf.ols("Price ~ Seating_Capacity", data=cars).fit
print(model_q32_linear.summary())
print(model_q32_linear.params)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Price   R-squared:
Model:                            OLS   Adj. R-squared:
Method:                 Least Squares   F-statistic:
Date:                Fri, 27 Mar 2026   Prob (F-statistic):
Time:                        02:52:10   Log-Likelihood:
No. Observations:                 976   AIC:                        2
Df Residuals:                     974   BIC:                        2
Df Model:                           1
Covariance Type:            nonrobust
==================================================================================
                       coef    std err          t      P>|t|      [0.0
----------------------------------------------------------------------------------
Intercept          4.39e+05   1.35e+05      3.263      0.001    1.75e+
Seating_Capacity  5.927e+04   2.63e+04      2.253      0.024    7641.1
==============================================================================
Omnibus:                      314.988   Durbin-Watson:
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
Skew:                           1.627   Prob(JB):                   1
Kurtosis:                       6.468   Cond. No.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
Intercept           439032.014102
Seating_Capacity     59267.998037
dtype: float64
```

The slope coeffieicient is about 59268. This means that each additional seat is associated with about a 59,268 rupee increase in price on average.

```
model_q32_cat = smf.ols("Price ~ C(Seating_Capacity)", data=cars).fit
print(model_q32_cat.summary())
print(model_q32_cat.params)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Price   R-squared:
Model:                            OLS   Adj. R-squared:
Method:                 Least Squares   F-statistic:
Date:                Fri, 27 Mar 2026   Prob (F-statistic):           (
Time:                        02:53:21   Log-Likelihood:
No. Observations:                 976   AIC:                        2
Df Residuals:                     971   BIC:                        2
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                               coef    std err          t      P>|t|
------------------------------------------------------------------------------
Intercept                  1.88e+05   3.64e+05      0.516      0.606
C(Seating_Capacity)[T.5]  5.465e+05   3.64e+05      1.500      0.134
C(Seating_Capacity)[T.6]  1.456e+06   5.15e+05      2.828      0.005
C(Seating_Capacity)[T.7]  7.359e+05   3.69e+05      1.996      0.046
C(Seating_Capacity)[T.8]  2.718e+05   3.99e+05      0.682      0.496
==============================================================================
Omnibus:                      324.467   Durbin-Watson:
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
Skew:                           1.663   Prob(JB):                   6
Kurtosis:                       6.618   Cond. No.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
Intercept                  1.880000e+05
C(Seating_Capacity)[T.5]    5.464839e+05
C(Seating_Capacity)[T.6]    1.456000e+06
C(Seating_Capacity)[T.7]    7.359487e+05
C(Seating_Capacity)[T.8]    2.718000e+05
dtype: float64
```

```
cars.groupby("Seating_Capacity")["Price"].mean()
```

Seating_Capacity	Price
4	1.880000e+05
5	7.344839e+05
6	1.644000e+06
7	9.239487e+05
8	4.598000e+05

dtype: float64

The difference are not linear to the specific point. Prices increase from 4 to 5 to 6 seats but then the pattern does not continue smoothly for 7 and 8 seats so the treating seating capacity is as categorical which shows that the relationship is only roughly linear at best.

Q3.3

```
cars["Age"] = 2026 - cars["Make_Year"]
cars[["Make_Year", "Age"]].head()
```

	Make_Year	Age
0	2017	9
1	2016	10
2	2019	7
3	2017	9
4	2017	9

```
def poly_cv_rmse(data, max_degree=10, seed=1):
    kf = KFold(n_splits=10, shuffle=True, random_state=seed)
    results = []

    for d in range(1, max_degree + 1):
        rmses = []
        formula = "Price ~ " + " + ".join([f"I(Age**{p})" for p in ra

        for train_idx, test_idx in kf.split(data):
            train = data.iloc[train_idx]
            test = data.iloc[test_idx]

            model = smf.ols(formula, data=train).fit()
            pred = model.predict(test)
            rmse = np.sqrt(np.mean((test["Price"] - pred) ** 2))
            rmses.append(rmse)

        results.append([d, np.mean(rmses)])

    return pd.DataFrame(results, columns=["degree", "cv_rmse"])

cv_results = poly_cv_rmse(cars, max_degree=10)
cv_results
```

	degree	cv_rmse
0	1	310378.396887
1	2	310457.787568
2	3	309616.738130
3	4	308930.145217
4	5	307509.787490
5	6	307682.560340
6	7	306600.440545
7	8	305047.474062
8	9	304339.367517
9	10	304510.067337

```
best_degree = cv_results.loc[cv_results["cv_rmse"].idxmin(), "degree"]
best_degree
```

```
np.int64(9)
```

usign 10 fold cross validation, the best model among degrees 1 thorugh 10 uses 9 poewrs of Age. It has the lowest avg RMSE.

Q3.4

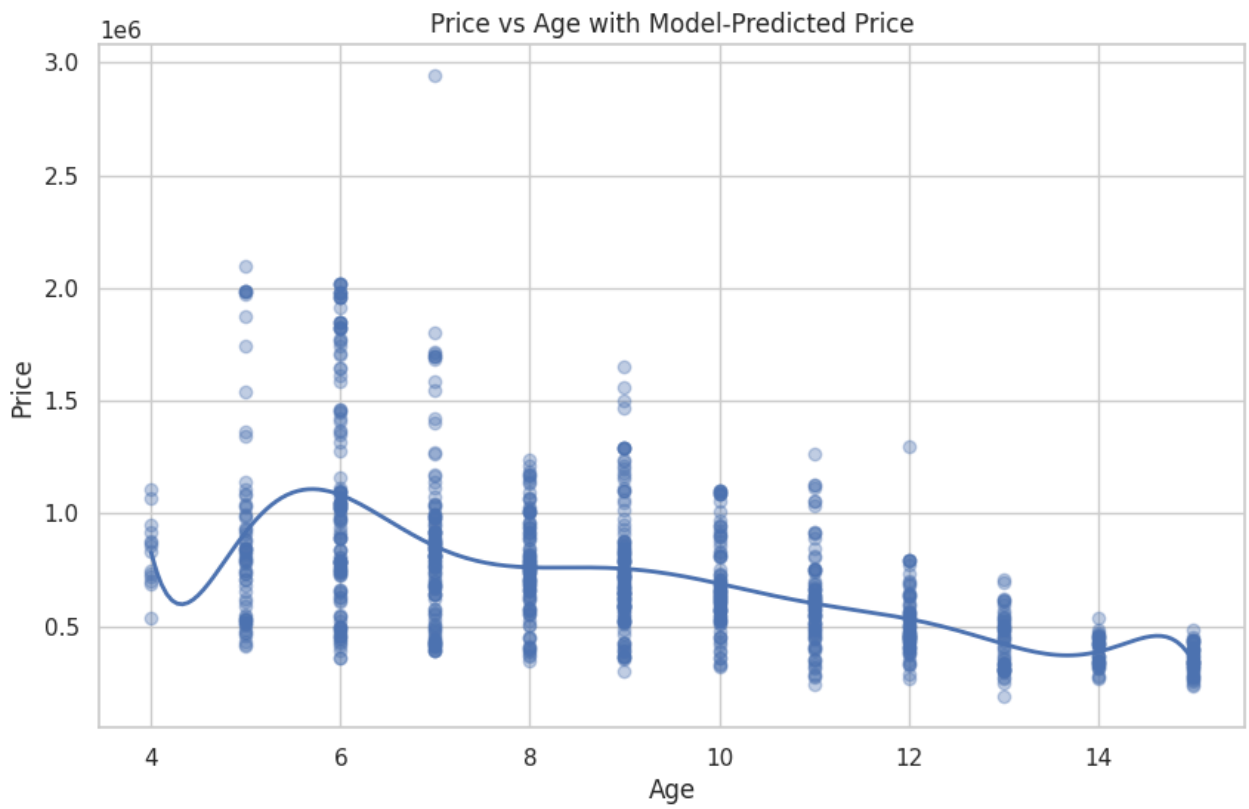
```
best_degree = 9
formula_best = "Price ~ " + " + ".join([f"I(Age**{p})" for p in range
best_model = smf.ols(formula_best, data=cars).fit()
print(best_model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Price   R-squared:
Model:                            OLS   Adj. R-squared:
Method:                 Least Squares   F-statistic:
Date:                Fri, 27 Mar 2026   Prob (F-statistic):          !
Time:                        03:04:27   Log-Likelihood:
No. Observations:                 976   AIC:                        2
Df Residuals:                     967   BIC:                        2
Df Model:                           8
Covariance Type:            nonrobust
==============================================================================
                  coef    std err          t      P>|t|      [0.025
------------------------------------------------------------------------------
Intercept    1.129e+09   3.69e+08      3.064      0.002    4.06e+08
I(Age ** 1) -1.255e+09   4.22e+08     -2.976      0.003   -2.08e+09
I(Age ** 2)  6.021e+08   2.09e+08      2.882      0.004    1.92e+08
I(Age ** 3) -1.637e+08   5.88e+07     -2.783      0.005   -2.79e+08
I(Age ** 4)  2.788e+07   1.04e+07      2.682      0.007    7.48e+06
I(Age ** 5)  -3.09e+06    1.2e+06     -2.584      0.010   -5.44e+06
I(Age ** 6)  2.232e+05   8.97e+04      2.490      0.013    4.73e+04
I(Age ** 7) -1.016e+04   4231.767     -2.400      0.017   -1.85e+04
I(Age ** 8)   264.5365    114.180      2.317      0.021      40.467
I(Age ** 9)    -3.0085      1.343     -2.240      0.025      -5.645
==============================================================================
Omnibus:                      252.448   Durbin-Watson:
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
Skew:                           1.243   Prob(JB):                   2
Kurtosis:                       6.763   Cond. No.                    !
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
[2] The condition number is large, 5.9e+14. This might indicate that tl
strong multicollinearity or other numerical problems.
```

```
age_grid = pd.DataFrame({"Age": np.linspace(cars["Age"].min(), cars["A
age_grid["predicted_price"] = best_model.predict(age_grid)

plt.figure(figsize=(10,6))
plt.scatter(cars["Age"], cars["Price"], alpha=0.35)
plt.plot(age_grid["Age"], age_grid["predicted_price"], linewidth=2)
plt.title("Price vs Age with Model-Predicted Price")
plt.xlabel("Age")
plt.ylabel("Price")
plt.show()
```



The fitted curve captures the general downward relationship between car price and age but there is still a lot of scatter around the curve so the model picks up the main trend but it does not fully explain the vast varaition in prices. This suggest age matters but other features also affect price a great amount.

Q4

```
heart = pd.read_csv("heart_hw.csv")
heart.head()
```

	Unnamed: 0	age	transplant	y
0	1	53	control	0
1	2	43	control	0
2	3	52	control	0
3	4	52	control	0
4	5	54	control	0

```
# proportions
control = heart[heart["transplant"] == "control"]["y"].mean()
treatment = heart[heart["transplant"] == "treatment"]["y"].mean()

ATE = treatment - control

control, treatment, ATE
```

```
(np.float64(0.11764705882352941),
 np.float64(0.34782608695652173),
 np.float64(0.23017902813299232))
```

Control = 0.117 Treat = 0.348 ATE = 0.230

receiving a transplant increases the probalbility of surviving three years by about 23.0 percent on average.

```
model_q42 = smf.ols("y ~ C(transplant)", data=heart).fit()
print(model_q42.params)
```

```
Intercept                     0.117647
C(transplant)[T.treatment]    0.230179
dtype: float64
```

Intercept = 0.1176 ---> survival probabliity for control group Transplant coeffiecient = 0.230 ----> exactly the ATE

```
model_q43 = smf.ols("y ~ C(transplant) + age", data=heart).fit()
print(model_q43.summary())
print(model_q43.params)
```

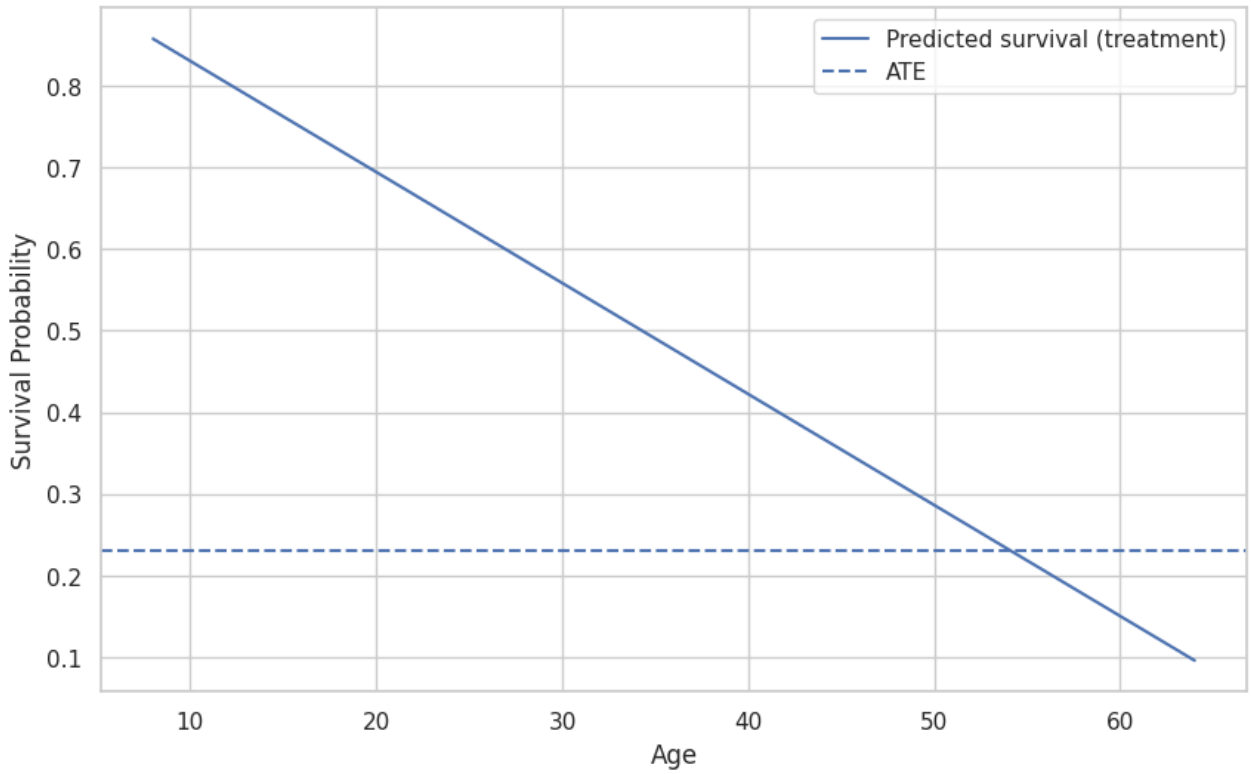
```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:
Model:                            OLS   Adj. R-squared:
Method:                 Least Squares   F-statistic:
Date:                Fri, 27 Mar 2026   Prob (F-statistic):           (
Time:                        03:24:18   Log-Likelihood:
No. Observations:                 103   AIC:
Df Residuals:                     100   BIC:
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                                coef    std err          t      P>|t|
------------------------------------------------------------------------------
Intercept                     0.7020      0.196      3.583      0.001
C(transplant)[T.treatment]    0.2647      0.088      3.006      0.003
age                          -0.0136      0.004     -3.204      0.002
==============================================================================
Omnibus:                       14.464   Durbin-Watson:
Prob(Omnibus):                  0.001   Jarque-Bera (JB):
Skew:                           0.753   Prob(JB):
Kurtosis:                       2.227   Cond. No.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
Intercept                    0.701957
C(transplant)[T.treatment]   0.264702
age                         -0.013607
dtype: float64
```

```
age_grid = pd.DataFrame({
    "age": np.linspace(heart["age"].min(), heart["age"].max(), 200),
    "transplant": "treatment"
})

age_grid["pred"] = model_q43.predict(age_grid)

plt.figure(figsize=(10,6))
plt.plot(age_grid["age"], age_grid["pred"], label="Predicted survival
plt.axhline(ATE, linestyle="--", label="ATE")
plt.xlabel("Age")
plt.ylabel("Survival Probability")
plt.legend()
plt.show()
```



After controlling for age the transplant coefficient decreases slightly showing that part of the original difference in suruvival rates was due to age differences between groups. The intercept shows the predicted survivival probabliity for a control patient at age 0. It also shows survival decrease as age increases.

So typically transplants help more for some ages than others to provide safe logenvity.

```
def cv_rmse_y(formula, data, k=10, seed=1):
    kf = KFold(n_splits=k, shuffle=True, random_state=seed)
    rmses = []

    for train_idx, test_idx in kf.split(data):
        train = data.iloc[train_idx]
        test = data.iloc[test_idx]

        model = smf.ols(formula, data=train).fit()
        pred = model.predict(test)
        rmse = np.sqrt(np.mean((test["y"] - pred) ** 2))
        rmses.append(rmse)

    return np.mean(rmses)

models = {
    "Model 1": "y ~ C(transplant)",
    "Model 2": "y ~ C(transplant) + age",
    "Model 3": "y ~ C(transplant)*age"
}

for name, f in models.items():
    print(name, cv_rmse_y(f, heart))
```

```
Model 1 0.43712414924209
Model 2 0.4209998366059898
Model 3 0.42010824697341675
```

Model 1 has the highest RMSE (worst performance). Model 2 and Model 3 both substantially improve predictive accuracy. Model 3 has the lowest RMSE, but the improvement over Model 2 is very small.

4.6

The model is based on observational data so it may be baised and not reflect true causual relation also it may reinforce existing inequalities in who recieves transplants. Such as age could effect unfraily disadvantage older patients. Using a model for life and death decision risks oversimplifying ethical choices.

This should not be used to decide transplant placement.

Many parts of the code was written using LLMs. I used them to guide and generate small chunks of code while remaining understanding of what is being produced.